

too are bytecodes which are interpreted by the JRE). A programming language which when compiled creates high performance code which can run on multiple architectures is deemed ideal for such usage. While Java is cross platform compatible, it's not faster than C/C++ for bit-level operations. Thus C/C++ are thought to be the best languages to write high performance ciphers with complicated mathematical procedures. As a side note, we would like to say that some people argue that Haskell is high performance as well, but benchmarks show that it's not faster than C/C++ (implementation of Haskell itself happens to be in C/C++) and thus it does not really outperform C/C++.

Since most modern ciphers deal with mathematical calculations, C/C++ would qualify easily as the duo comes closest to hardware while maintaining high speed for calculations.

A missing piece

We told you earlier that we were not going to illustrate the best ciphering technique. However the algorithm that we have designed has a missing piece in it: the cipher will fail for any data whose size is not a direct multiple of 128 bits (16 bytes). There are multiple ways to do just that. One of those ways would be to pad the plaintext with random data to make its size a perfect multiple of 128 bits and send along the size of data that's actually to be considered after decrypting along with the key.

Disadvantages of using your own cipher

Nothing comes without its problems. A custom-designed cipher such as the one we illustrated in this chapter are no exceptions.

1. The first disadvantage of making your own cipher is that it's going to be weak (unless you're a mathematical whizkid). Well established ciphers are the result of years (if not decades) of mathematical research. It's next to impossible to design a new algorithm overnight that would be cleverer than years of research done by actual mathematics whizzes.
2. The second disadvantage is that no one except you would be able to use it for practical purposes. The software to encrypt and decrypt data using well known algorithms is usually already installed in the operating system or other software which is needed to use them (e.g. web browsers). Your algorithm is going to be just yours. You know the design and how it's implemented, and no one else can make use of it. Not only that, you need to transfer the algorithm to everyone who wants to use it.